The 10 MDS Components

Summary:

This guide provides definitions, examples and common implementations for **10 key components** in modern data architectures.

How to Use This:

While there are many variations of each in the real-world, use this guide **as a baseline** for understanding where, why and how each component is used. From there, you can decide if it's an area you need to learn more about or perhaps add to an existing architecture.

The List:

Essentials:

1. Storage & Databases
2. Ingestion
3. Transformation
4. Visualization
5. Version Control & CI/CD

Advanced:

1. Orchestration
2. Containers
3. Infrastructure as Code (IaC)
4. Data Quality
5. Reverse ETL

---

1. Storage & Databases

Store vast amounts of data objects from all of your source systems, in any format, in a cloud server.

How it's implemented

Storage:

- Centralized landing zone (data lake)
- Split by "temperature" to manage access & cost (hot, warm, cold)
- Data backups

Databases:

- Backend of applications
- Centralized landing zone (data lake)
- Single-source of truth & business logic (data warehouse)
- Options to handle unstructured & structured data

Why it's helpful

Storage:

- Scalability
- Low Cost
- Flexibility (file types)
- Availability

Databases:

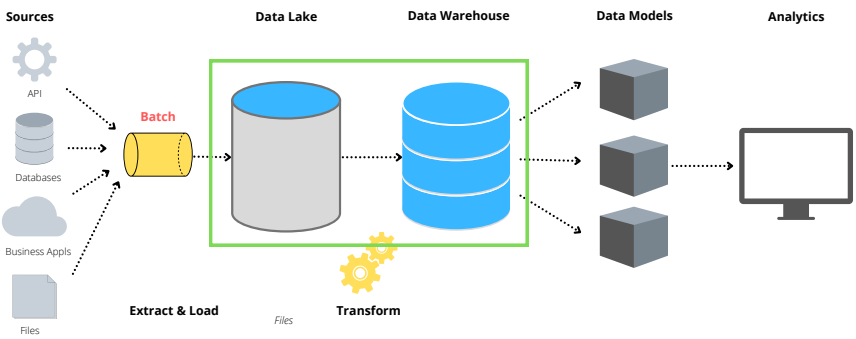
- Organization & structure
- Well-defined & understood query languages (ex. SQL)
- Performance
- Security
- Connect with other tools as the main hub of all data

Example Tools

1. S3, Azure BLOB, Google Storage
2. Snowflake, Databricks, BigQuery
3. MySQL, Postgres, SQL Server
4. MongoDB, CassandraDB

Languages & File Types

1. SQL, Python
2. CSV, XML, Parquet, JSON



---

2. Ingestion

Extract data from various sources and load it into a consolidated landing zone such as a data lake.

How it's implemented

- Batch (scheduled) vs Streaming (real-time)
- Tools with pre-built connectors for easy connection to sources & destinations
- Custom scripts (ex. python)
- Load into a data lake or temporary "staging" area (ETL vs ELT)

Why it's helpful

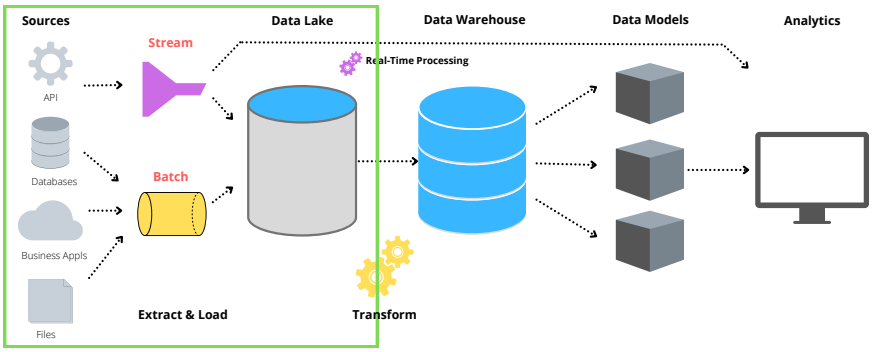
- Allows data across different systems to be fed into your own environment
- Provides the ability to centralize the data (ex. in storage or a database)
- Sets the foundation for further insights

Example Tools

1. Fivetran, Stitch, Airbyte
2. Apache Kafka, Kinesis, Debezium
3. Custom Python scripts

Languages & File Types

1. SQL, Python
2. CSV, XML, Parquet, JSON, YAML



---

3. Transformation

The process of turning raw data (loaded from extract phase) into clean, organized & useful data that's ready for analytics

How it's implemented

- Occurs after data is already loaded (via extract & load)
- Or could occur during ingestion (streaming)
- Code-based tools (ex. dbt)
- Custom scripts (python, stored procedures)
- UI-based drag & drop tools
- Run on a schedule separately from extract & loading (ELT)
- Big data processing (ex. Spark)

Why it's helpful

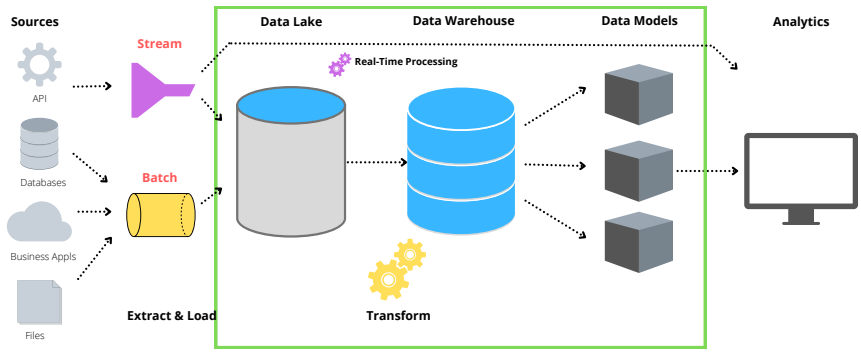
- Raw data by itself isn't always helpful
- Applies real-world rules and concepts to raw data so it makes more sense
- Provides organization and consistency so you can have more confidence

Example Tools

1. dbt
2. Apache Spark
3. AWS Glue, Azure Data Factory

Languages & File Types

1. SQL, Python, Jinja
2. CSV, XML, Parquet, JSON, YAML



---

4. Reporting & Analytics

Provides insights to the transformed data to make better decisions and have clarity on the business

How it's implemented

- Data visualization tools connect to the database (warehouse/marts)
- Additional logic sometimes added within the tool as well (more logic)

Why it's helpful

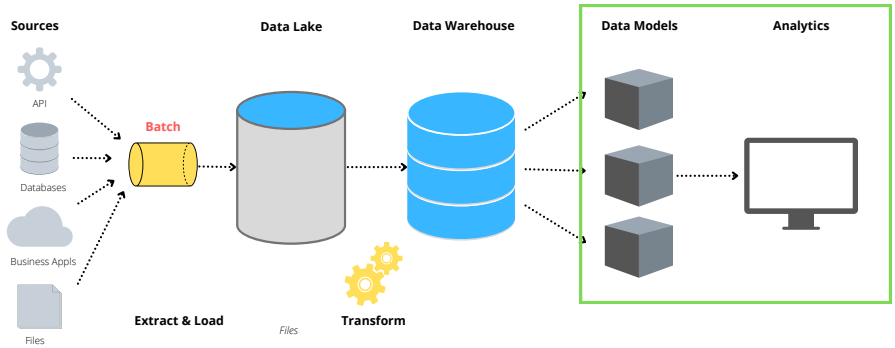
- Provides insights to the business operations
- Makes predictions
- Analyze history
- Facilitates decision making
- User friendly, abstracts away most of the complex engineering behind the scenes

Example Tools

1. Power BI, Looker, Tableau
2. Metabase, Superset
3. ML Tools

Languages & File Types

1. SQL, Python, R



---

5. Version Control & CI/CD

A way to monitor all changes to a development project. A form of automation that is based on events and triggers vs schedules to release and test code or complete some other task

How it's implemented

- Hosted platforms that allow you to connect a development project
- Requires a local software (git) which is often available by default
- Automation through the git platform or using another (orchestration tool)

Why it's helpful

Version Control

- Easier to revert changes
- More transparency
- Avoid conflicts
- Provides a good workflow for development

CI/CD

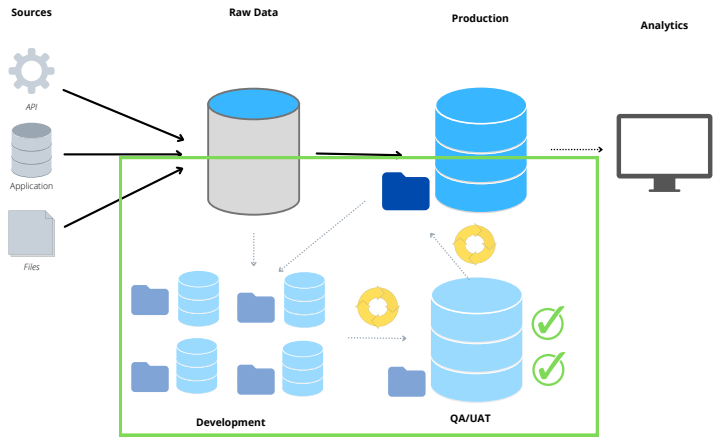
- Faster to production
- Removes manual tasks (saves time)
- More iterative

Example Tools

1. GitHub
2. GitLab
3. BitBucket

Languages & File Types

1. Bash/Zsh, Python
2. YAML



---

6. Orchestration

A way to connect and trigger different aspects of your pipeline through a single tool. This could also be an orchestration for a single component, for example containers

How it's implemented

- Separate tools focused on orchestration
- Alternatively it could be part of an existing tool you have (ex. database server)
- Could also be custom scripts (ex. Python)

Why it's helpful

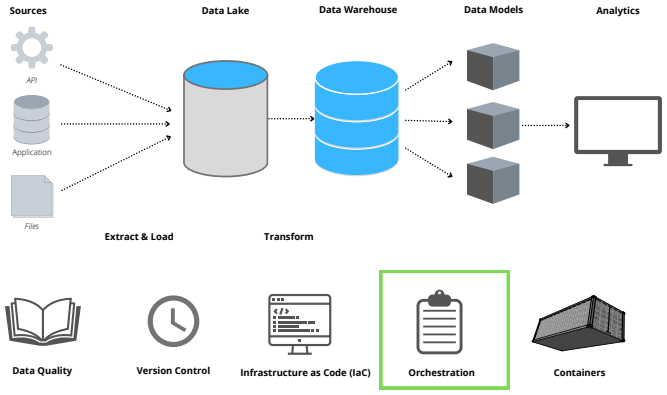
- Without a central orchestrator, your process can become hard to manage and feel overwhelming
- It helps you stay monitor and control at a high level in a central location

Example Tools

- Airflow, Prefect, Dagster, Luigi, Jenkins
- Kubernetes, Openshift
- SQL Agent Job, SSIS, hosted servers

Languages & File Types

1. Python, Bash/Zsh
2. YAML



---

7. Containers

An isolated "virtual" instance of an operating system that can be used to run applications and processes

How it's implemented

- Create a specific run-time environments for automated processes
- Hosting open-source tools
- Creating development environments

Why it's helpful

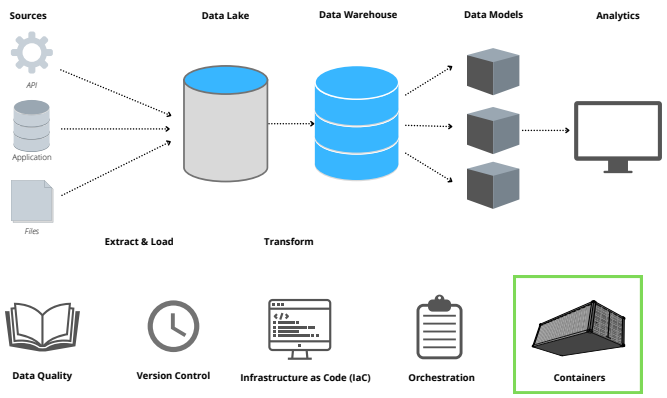
- Allows you to easily create a specific type of environment to run what you need without the overhead of creating entirely new machines
- Can be easily created and dropped for automated processes

Example Tools

1. Docker

Languages & File Types

1. Bash/Zsh
2. DockerFile, YAML



---

8. Infrastructure as Code (IaC)

A way to create and manage cloud resources with version-controlled code, rather than manually

How it's implemented

- Automate deployments, updates, on-boarding, etc.
- Manage roles, permissions & configurations for cloud platforms
- Often written in YAML or JSON

Why it's helpful

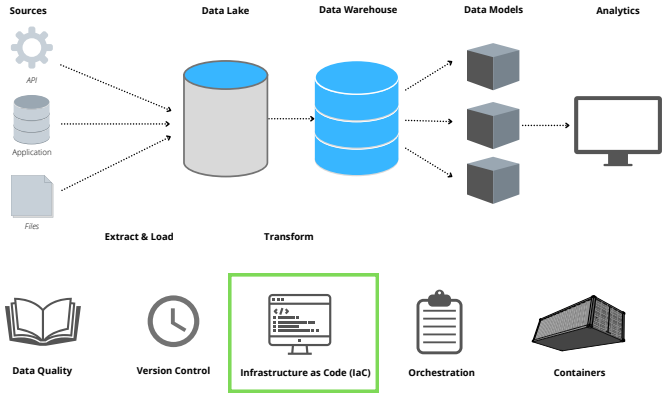
- Reduces manual steps, provides transparency and improves maintenance
- Can be part of an overall pipeline or manually triggered
- Ensures consistency

Example Tools

1. Terraform
2. Ansible
3. AWS Cloud Formation, Azure Resource Manager, GCP Deployment Manager

Languages & File Types

1. Bash/Zsh, Python
2. YAML, Terraform Files



---

9. Data Quality

Ensure data accuracy, completeness, consistency, and reliability

How it's implemented

- Automated testing at various steps in a pipeline
- Code linters or automated code review tools
- Data contracts with data producers (ex. software engineers)
- Documentation & lineage
- Data catalogs to organize all metadata for an organization

Why it's helpful

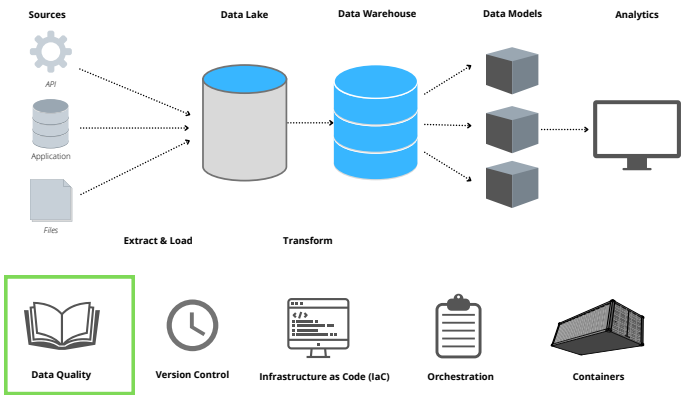
- Improves decision making and confidence in the data
- Helps catch errors before reaching end users
- Provides consistency and therefore more efficiency in development
- Provides Lineage

Example Tools

- dbt tests
- Great Expectations
- SQL Fluff, PyLint
- Data Catalog: DataHub, Atlan, Castor, Collibra
- Data Observability: Monte Carlo, Amundson

Languages & File Types

1. SQL, Python, Bash/Zsh
2. YAML



---

10. Reverse ETL

Syncing data from a central data warehouse back to operational tools (ex. SaaS tools) where the new custom fields would be useful.

How it's implemented

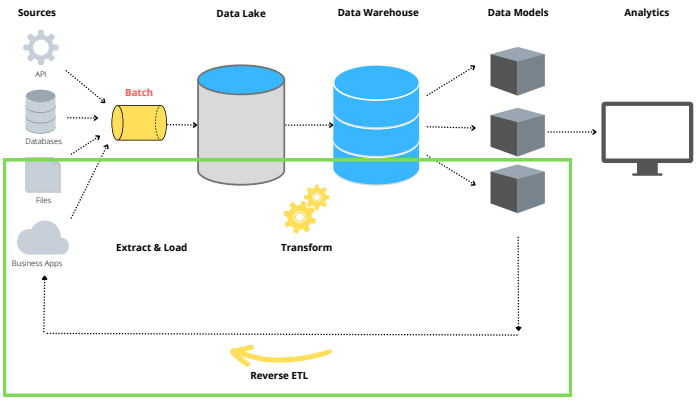
- Often at the "end" of the data pipeline
- Sends data from data warehouse back to business applications
- Triggered after warehouse is updated or run on a separate schedule

Why it's helpful

- It helps business users stay in the tools they know while still providing business-approved custom fields.
- Offloads technical requirements for this type of information to get back to business users
- Avoids the need for extra custom reports
- Avoid conflicting data between apps

Example Tools

1. Census
2. Hightouch
3. Rudderstack

Languages & File Types

1. SQL, Bash/Zsh
2. YAML